Connor Villanueva, Timothy Matthies, Jazzy De Los Santos
Prof. Foaad Khosmood
CSC482: Natural Language Processing
20 March 2026

Final Report

In our final project, the main problem we are trying to tackle is the vagueness and complexity of idioms and pragmatics in conversational English. Idioms and pragmatics are often complicated by the years of cultural, societal, and historical context behind them. In a direct coherent English sentence, context has typically been an issue for engineers in the NLP (Natural Language Processing) field. When introducing idioms into a sentence, this also introduces a whole new layer of complexity to its context. Not only does one have to identify what part of the sentence is an idiom, they also have to derive the true meaning within the sentence and know it is different from the literal words of the idiomatic phrase.

An example of an idiom is: “She decided to hit the books before her big exam”. The idiomatic phrase, “hit the books”, does not mean that the subject is literally going to hit some books. Instead, its true meaning is that she’s going to study. One of the most common issues in NLP is serving a phrase or sentence’s true meaning versus its literal meaning. With a starter database of idioms derived from Doughty (2017), in this project, we sought to bridge this gap between the literal and true in idioms and pragmatics. To do so, we identified an idiom in and outside of a sentence, connected it to its true meaning, and also replaced the idiomatic phrase (if within a sentence) with its definition, preserving grammar and tense, to create a new sentence with its original meaning but without metaphor or simile.

Our starting database of idioms is derived from a .json file provided by the work of the master’s thesis *Quantifying the Impact of Idiom Recognition in Genre Classification* by Jon Doughty (2017). From this json, we created our own parquet file that was cleaned and proofed for any faulty data. This data contained the following columns: variations, idiom, usage, sources, definition, id, and confidence. Of these columns, variations, usage, and definition could all be an empty list or empty string. We created a few versions of the idiom parquet to handle some of these blanks and to also improve the robustness and handling of some of the functions in our code. The finalized version contained variations of the idiom in its singular and present tense form and definitions for every idiom. Both of these were necessary for our code’s functionality.

Within our code, the main libraries we utilized were spaCy for natural language processing tasks such as tokenization, phrase matching, and identifying idioms within sentences. We used DuckDB to efficiently store, query, and manipulate the idiom dataset, which allowed for fast access to definitions and variations. RapidFuzz was used for handling approximate string matching and improving idiom detection robustness. Python’s difflib was also used for string matching and helped with minor variations and inconsistencies in the user input. NLTK was used for additional language resources and other testing.

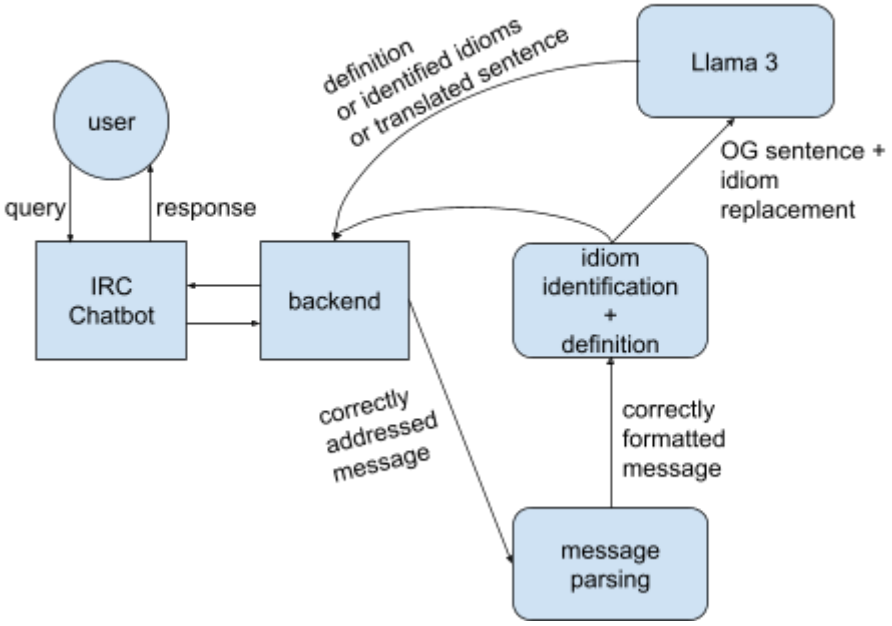
We also made use of requests to interface with a locally hosted LLM, Llama3 via Ollama, for grammar correction and sentence rewriting. This was used to refine sentence replacements and

improve grammatical correctness after substituting idioms with their definitions. Finally, we also relied on standard Python libraries, such as re for regular expression, socket, select, and time for the IRC-based chatbot, which became the interface layer for users to query the idiom parser and translator.

The system is set up as a simple pipeline that connects the IRC chatbot to a backend that does parsing, identification, and replacement work. A user sends a message to the IRC chatbot and then that message gets passed to the backend where it's parsed to figure out exactly what the user is asking. From there, if it's a valid message or request, the system checks if the request is about finding idioms, defining one, or replacing idioms in a sentence. Once it is verified that the message is correctly formatted for its request, it's passed to the idiom identification and definition parser which uses both rule-based and similarity-based approaches to detect idioms and retrieve their meanings.

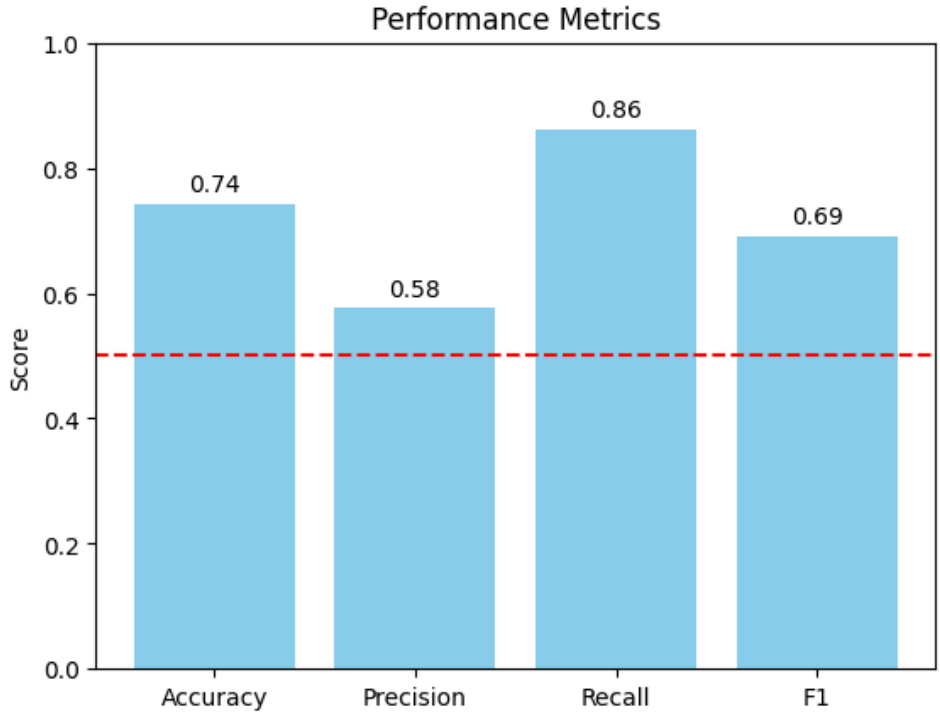
After an idiom is identified, its definition is pulled from the dataset and substituted back into the original sentence. At this point, the sentence is often technically correct but can sound awkward or out of place because the idiom definitions are not always written to naturally fit into a sentence, especially with all the varying possible verb tenses. To fix this, the modified sentence is passed into the Llama 3 model with Ollama, which then smooths out the grammar and adjusts the wording so it sounds natural while preserving as much of the original meaning as possible. This step was particularly necessary when dealing with tense or phrasing issues that the simple substitution could not handle.

Once the sentence is modified and cleaned, or if the user requested other information, the result is sent back "through" the backend to the chatbot and then to the user. Overall, the system is designed to be straightforward and a lot of the work is handled in stages with each part focusing on specific tasks like parsing, identifying, replacing, refining, and communicating.



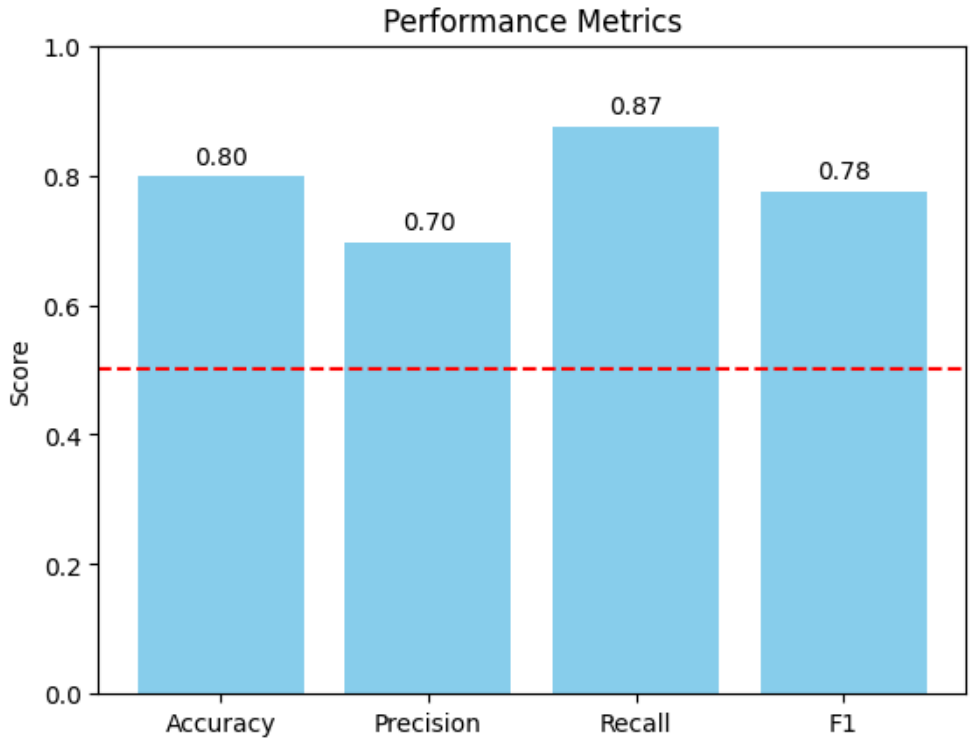
For our evaluation, we benchmarked our model's abilities on two versions. Our first version would pick out the best possible idiom that matched with one identified in the input and the second version transformed the input into its canonical form for more robust matching to the database of which we included variations of an idiom's canonical form. These two versions were tested against a combination of two documents. One document contained sentences with idioms and the other contained sentences without idioms. Since one document was longer than the other by a substantial amount, we made sure that an equal amount from both documents were included in the evaluation. With this in mind, the versions were tested against at least 400 lines of both idiomatic and non-idiomatic phrases.

Version 1, which did not utilize the canonical form transformation, performed relatively well in idiom classification. For accuracy, precision, recall, and F1 score, it scored 0.74, 0.58, 0.86, and 0.69, respectively. When compared to a score of 0.5, which would be equivalent to a model randomly classifying/identifying an idiom or lack of one, it performed better. The main score that we noticed performed the most poorly was the precision. This meant that version 1 was classifying most sentences as containing an idiom even when there wasn't one, producing a lot of false positives.



Version 2, whose only change was the addition of the canonical form transformation, showed a slight but clear improvement across the board. Accuracy went from 0.74 to 0.80, precision went from 0.58 to 0.70, recall went from 0.86 to 0.87, and the F1 score went from 0.69 to 0.78. With improvements ranging from 0.01 to 0.12, this added feature boosted performance on a noticeable level. Precision showed the biggest jump of +0.12 and a final score of 0.70. Seeing as this was

our metric that we were the most unhappy with, these improved metric scores indicated a correct step.



Some of our biggest shortcomings dealt with both identifying idioms and properly replacing them in context. One feature we had planned but did not end up implementing was detecting idioms that were not already in our database. Because of this, the chatbot effectively "fails" when it encounters an unfamiliar idiom. Instead of attempting a best guess, it simply tells the user it cannot help. Even within our fairly large dataset, the definitions themselves were not always ideal. Many entries were written as dictionary-style explanations rather than sentence-ready replacements. For example, an idiom like "more Catholic than the Pope" has a definition that is technically correct but too formal and rigid to drop cleanly into a sentence, which creates awkward or unnatural outputs.

A larger issue comes from how idioms behave in real language. Idioms are not always used in a fixed form. They vary in tense, structure, and placement, so much of our work went into normalizing them into a canonical form before matching. We also incorporated an LLM to help preserve grammar during replacement, but this approach still has clear limitations. In particular, negation and hedging cases consistently break the system. For example, a sentence like "She's not over the moon about it" can end up being translated into something like "She's not extremely happy about it," which loses the nuance and natural tone of the original. In more extreme cases, the system can produce outputs that are outright incorrect or ungrammatical, especially when definitions do not align cleanly with the sentence structure. Overall, these issues show that

idioms cannot be treated as simple word substitutions. They are highly context-dependent, and handling them correctly requires a deeper understanding of both grammar and pragmatics.

Overall, this project showed that identifying and translating idioms in natural language is both possible and useful. However, it also showed that it's much more difficult than the problem initially seems. While our system was able to correctly identify and replace many idioms, especially with our addition of canonical form transformations, there were still clear limitations in handling a lot of the more complex cases. The combination of rule-based matching, similarity, and LLM based grammar correction worked well for a large portion of the inputs. The improvement from version 1 to version 2 demonstrated that even small changes in preprocessing can have a noticeable impact on performance. However, the system still struggles with edge cases such as some negation, variations in phrasing, and definitions that do not naturally fit into sentences even with the LLM corrections.

For potential future work, one of the biggest improvements would be expanding beyond the current fixed idiom database. Adding the ability to detect and infer unknown idioms by possibly using more advanced language models or embeddings would make the system more robust. Another improvement could be refining the replacement process by potentially introducing a "sentence replacement" field for more natural sentence outputs instead of relying on the raw dictionary definitions from sources provided in the idiom dataset. Also, improving handling of things like negation or contextually sensitive or ambiguous phrases would reduce the amount of incorrect, awkward, or overly robust or literal translations. Finally, the system could either be a more flexible IRC chatbot interface or it could be extended beyond into another interface like a web app or API. This would make it easier to eventually scale and to improve accessibility, which is the heart of the motivation behind this project.

References

Doughty, J. (2017). *Quantifying the impact of idiom recognition in genre classification* (Master's thesis, California Polytechnic State University, San Luis Obispo).